

# Projektdokumentation

LF8 – Daten systemübergreifend bereitstellen

Projekt: Lernmaterialverwaltung

Autor: Auric Ergeson

Datum: Mai 2026

*Berufsbildende Schulen Verden – Anwendungsentwickler*

# Inhaltsverzeichnis

|                                                  |    |
|--------------------------------------------------|----|
| 1. Projektauftrag .....                          | 3  |
| 1.1 Ausgangssituation .....                      | 3  |
| 1.2 Aufgabenstellung.....                        | 3  |
| 1.3 Projektumfeld .....                          | 3  |
| 2. Projektplanung .....                          | 3  |
| 2.1 Lösungsansatz und Alternativen.....          | 3  |
| 2.2 Zeitplanung .....                            | 4  |
| 3. Projektdurchführung.....                      | 4  |
| 3.1 Datenbankdesign .....                        | 4  |
| 3.2 Implementierung.....                         | 4  |
| 3.3 Ergebnisse .....                             | 5  |
| 4. Funktionsübersicht.....                       | 7  |
| 5. Projektabschluss.....                         | 7  |
| 5.1 Soll-Ist-Vergleich .....                     | 7  |
| 5.2 Zeitaufwand-Vergleich.....                   | 7  |
| 5.3 Persönliche Reflexion.....                   | 8  |
| 6. Quellenverzeichnis.....                       | 8  |
| Anhangsverzeichnis.....                          | 9  |
| Anhang I: SQL-Schema .....                       | 10 |
| Anhang II: ERD und ERM .....                     | 13 |
| Entitäten und Beziehungen .....                  | 13 |
| Anhang III: Normalisierte Tabellenstruktur ..... | 15 |
| Anhang IV: Implementierungsdetails.....          | 18 |
| Architektur .....                                | 18 |
| Verwendete Bibliotheken.....                     | 18 |
| Speicherstrategie .....                          | 18 |
| Die sieben Datenbankabfragen .....               | 18 |
| Quellcode-Kommentare .....                       | 18 |
| Anhang V: Installationsanleitung .....           | 20 |
| Schritt 1 – Abhängigkeiten installieren.....     | 20 |
| Schritt 2 – .env Datei anlegen .....             | 20 |
| Schritt 3 – Datenbank einrichten.....            | 20 |
| Schritt 4 – Anwendung starten .....              | 20 |

# 1. Projektauftrag

## 1.1 Ausgangssituation

An unserer Berufsschule wurden Lernmaterialien bisher hauptsächlich per E-Mail weitergegeben oder in verschiedenen Ordnern auf Schulrechnern abgelegt. Ein Lehrer hatte zum Beispiel Unterlagen für dasselbe Thema in drei verschiedenen Verzeichnissen gespeichert – ohne einheitliche Benennung und ohne dass die Azubis wussten, welche Version aktuell war.

Im Rahmen von Lernfeld 8 wurde mir die Aufgabe gestellt, eine datenbankgestützte Lösung zu entwickeln, die dieses Problem systematisch löst. Ich habe das Projekt vom 06.04.2026 bis zum 16.05.2026 als Einzelprojekt durchgeführt.

Ein Problem war von Anfang an klar: Die Lösung muss einfach zu bedienen sein, weil nicht alle Nutzer technisch erfahren sind. Deshalb habe ich mich für eine konsolenbasierte Anwendung entschieden, die ohne grafische Oberfläche auskommt.

## 1.2 Aufgabenstellung

Das System sollte es ermöglichen, Lernmaterialien zentral zu speichern, zu suchen und herunterzuladen. Zusätzlich waren folgende technische Anforderungen vorgegeben: MySQL als Datenbank, Python als Programmiersprache, mindestens sieben SQL-Abfragen mit unterschiedlichen JOIN- und Aggregationstypen sowie eine duale Speicherstrategie für Dateien.

Ich habe die Anforderungen in acht konkrete Funktionen übersetzt, die alle über ein Hauptmenü erreichbar sind. Die genaue Auflistung zeigt Kapitel 4 – Funktionsübersicht.

Das Projekt wurde vollständig eigenständig als Einzelarbeit durchgeführt. Konzeption, Datenbankdesign, sämtlicher Python-Code und die Dokumentation stammen von mir. Externe Leistungen wurden nicht in Anspruch genommen.

## 1.3 Projektumfeld

Das Projekt war ein Einzelprojekt. Ich habe es auf meinem privaten Laptop entwickelt, auf dem MySQL 8.0, Python 3.12 und Visual Studio Code installiert sind. Die Datenbankmodellierung habe ich mit DBeaver durchgeführt, das auch das ERD-Diagramm erzeugt hat.

Als Versionskontrolle habe ich Git verwendet. Schnittstellen zu anderen Systemen gibt es nicht – die Anwendung läuft vollständig lokal. Die einzige externe Abhängigkeit ist die MySQL-Datenbank, die auf demselben Rechner läuft.

*(ERD-Diagramm: siehe Anhang II)*

# 2. Projektplanung

## 2.1 Lösungsansatz und Alternativen

Zunächst war unklar, ob ich eine webbasierte Anwendung oder eine Kommandozeilenanwendung entwickeln soll. Eine Webanwendung mit Flask wäre benutzerfreundlicher gewesen, hätte aber deutlich mehr Entwicklungszeit benötigt und war für den vorgegebenen Zeitraum nicht realistisch.

Ich habe mich deshalb bewusst für die Kommandozeilenlösung entschieden. Der Vorteil: Ich konnte mich auf die Datenbanklogik konzentrieren, die der eigentliche Kern der Aufgabe war. Python bietet mit SQLAlchemy eine mächtige ORM-Bibliothek, die mir das Arbeiten mit der Datenbank deutlich erleichtert hat.

Bei der Datenbankstruktur hatte ich zunächst überlegt, Tags als kommaseparierte Liste direkt in der material-Tabelle zu speichern. Das wäre einfacher gewesen, hätte aber gegen die erste Normalform verstoßen. Ich habe mich deshalb für eine separate tag-Tabelle mit Verbindungstabelle material\_tag entschieden.

## 2.2 Zeitplanung

Das Projekt lief über sechs Wochen (06.04.2026 – 16.05.2026). Die folgende Tabelle zeigt die geplanten und tatsächlich benötigten Stunden pro Arbeitspaket.

*(Zeitvergleich mit Abweichungen: siehe Kapitel 5.2)*

| Phase                        | Geplant     | Tatsächlich | Abweichung  |
|------------------------------|-------------|-------------|-------------|
| Analyse & Planung            | 4 h         | 5 h         | +1 h        |
| DB-Design (ERD, ERM, Schema) | 6 h         | 8 h         | +2 h        |
| Python-Implementierung       | 15 h        | 18 h        | +3 h        |
| Testen & Debugging           | 4 h         | 5 h         | +1 h        |
| Dokumentation                | 6 h         | 7 h         | +1 h        |
| <b>Gesamt</b>                | <b>35 h</b> | <b>43 h</b> | <b>+8 h</b> |

## 3. Projektdurchführung

### 3.1 Datenbankdesign

Bevor ich mit dem Programmieren angefangen habe, habe ich zuerst das Datenbankschema entworfen. Das war eine bewusste Entscheidung: Wenn die Datenbankstruktur nicht stimmt, zieht sich das durch den gesamten Code.

Ein Problem beim Design war die Frage der Dateispeicherung. Ich habe mich für eine duale Strategie entschieden: Dateien unter 1 MB werden direkt als LONGBLOB in der Datenbank gespeichert, größere Dateien werden im Ordner uploads/ abgelegt. Das vollständige Schema ist in Anhang I dokumentiert, das ERD-Diagramm in Anhang II.

Die Normalisierung habe ich nach den drei Normalformen durchgeführt. Besonders die Entscheidung, Tags in eine eigene Tabelle auszulagern, war wichtig für die 1NF. Die vollständige Tabellenstruktur mit allen Spalten und Datentypen ist in Anhang III dargestellt.

*(SQL-Schema: siehe Anhang I – ERD: siehe Anhang II – Tabellenstruktur: siehe Anhang III)*

### 3.2 Implementierung

Die Implementierung habe ich in drei Phasen aufgeteilt: zuerst database.py (Datenbankverbindung und Tabellen-Mapping), dann services.py (alle

Datenbankfunktionen), zuletzt main.py (das Konsolenmenü). Diese Reihenfolge hat sich bewährt, weil ich immer auf fertigen Schichten aufgebaut habe.

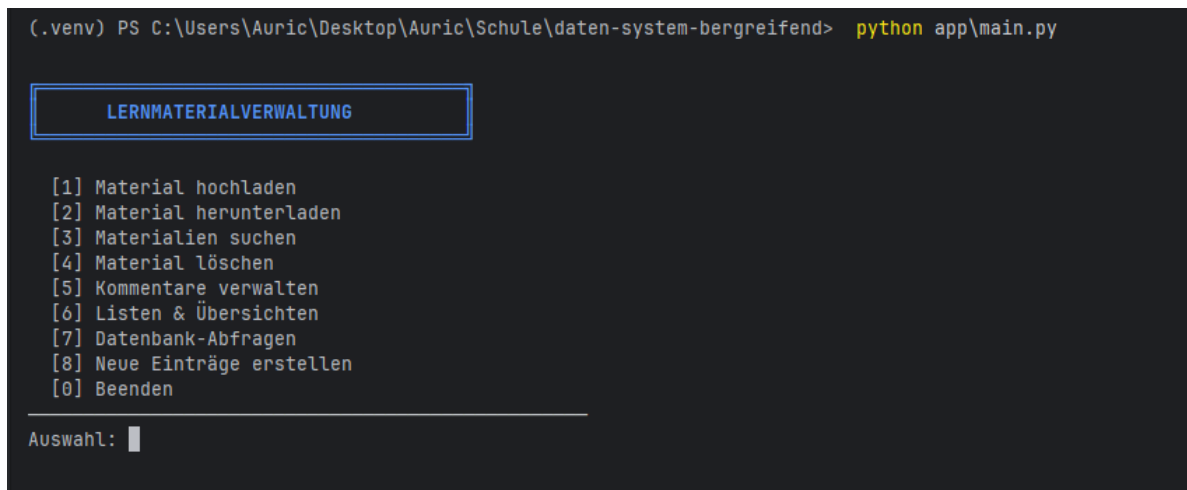
Ein Problem war anfangs die SQLAlchemy automap-Funktion. Ich wusste nicht, dass die Tabellen zuerst in der Datenbank existieren müssen, bevor automap\_base() sie einlesen kann. Der Fehler beim ersten Test war nicht sofort verständlich – nach etwas Recherche in der Dokumentation war die Ursache aber klar. Die Lösung war einfach: Schema zuerst importieren, dann die Anwendung starten.

Die sieben SQL-Abfragen habe ich alle in services.py implementiert. Abfrage 7 ist die komplexeste: Sie verbindet vier Tabellen (material, benutzer, themengebiet, kategorie) mit drei JOINS und unterstützt optionale WHERE-Filter. Die Architektur und alle sieben Abfragen sind in Anhang IV ausführlich dokumentiert.

*(Architektur und SQL-Abfragen: siehe Anhang IV)*

### 3.3 Ergebnisse

Nach 43 Stunden Entwicklungsarbeit habe ich eine vollständig funktionsfähige Kommandozeilenanwendung fertiggestellt. Ich habe die Anwendung während des Testens bewusst mit echten Daten durchgespielt: Mehrere Materialien hochgeladen, wieder heruntergeladen, kommentiert und gelöscht – alles hat funktioniert. Abbildung 2 zeigt das Hauptmenü beim Start der Anwendung.



```
(.venv) PS C:\Users\Auric\Desktop\Auric\Schule\daten-system-bergreifend> python app\main.py

LERNMATERIALVERWALTUNG

[1] Material hochladen
[2] Material herunterladen
[3] Materialien suchen
[4] Material löschen
[5] Kommentare verwalten
[6] Listen & Übersichten
[7] Datenbank-Abfragen
[8] Neue Einträge erstellen
[0] Beenden

Auswahl: █
```

Abbildung 2: Hauptmenü der Lernmaterialverwaltung

Die Materialliste (Menüpunkt 6 → Materialien) gibt alle gespeicherten Einträge als formatierte Tabelle aus – mit ID, Titel, Dateityp, Größe und der Information, ob die Datei direkt in der Datenbank oder als Dateipfad gespeichert ist. Die Ausgabe in Abbildung 3 zeigt echte Testdaten aus der Datenbank.

```
(.venv) PS C:\Users\Auric\Desktop\Auric\Schule\daten-system-bergreifend> python app\main.py
```

LERNMATERIALVERWALTUNG

```

[1] Material hochladen
[2] Material herunterladen
[3] Materialien suchen
[4] Material löschen
[5] Kommentare verwalten
[6] Listen & Übersichten
[7] Datenbank-Abfragen
[8] Neue Einträge erstellen
[0] Beenden

```

Auswahl: 6

— LISTEN —

```

[1] Materialien
[2] Benutzer
[3] Themengebiete
[4] Kategorien
[5] Tags
[0] Zurück

```

Auswahl: 1

*Materialien*

| ID | Titel           | Dateiname                                                                           | Typ             | Größe    | In DB |
|----|-----------------|-------------------------------------------------------------------------------------|-----------------|----------|-------|
| 1  | test            | test.pdf                                                                            | application/pdf | 0.00 MB  | Ja    |
| 2  | test2           | fastapi-modern-python-web-development-1nbsped-1098135504-9781098135508_compress.pdf | application/pdf | 24.03 MB | Nein  |
| 3  | Test second     | jsonformatter.txt                                                                   | text/plain      | 0.11 MB  | Ja    |
| 4  | Test PDF        | test_small.pdf                                                                      | application/pdf | 0.00 MB  | Ja    |
| 5  | Größe Testdatei | test_large.txt                                                                      | text/plain      | 1.26 MB  | Nein  |

— LISTEN —

```

[1] Materialien
[2] Benutzer
[3] Themengebiete
[4] Kategorien
[5] Tags
[0] Zurück

```

Auswahl: █

Abbildung 3: Materialliste mit rich-Tabellenformatierung

Alle sieben SQL-Abfragen laufen stabil. Abbildung 4 zeigt das Ergebnis von Abfrage 1 (COUNT: Anzahl der Materialien je Themengebiet) – direkt aus der laufenden Anwendung. Die komplexeren Abfragen, insbesondere Abfrage 7 mit Multi-JOIN und optionalen Filtern, liefern ebenfalls korrekte Ergebnisse.

LERNMATERIALVERWALTUNG

```

[1] Material hochladen
[2] Material herunterladen
[3] Materialien suchen
[4] Material löschen
[5] Kommentare verwalten
[6] Listen & Übersichten
[7] Datenbank-Abfragen
[8] Neue Einträge erstellen
[0] Beenden

```

Auswahl: 7

— DATENBANK-ABFRAGEN —

```

[1] Materialien pro Themengebiet (COUNT)
[2] Ø Dateigröße pro Dateityp (AVG)
[3] Materialien + Themengebiete (JOIN)
[4] Materialien + Kommentare (JOIN)
[5] Kommentare pro Material (JOIN + COUNT)
[6] Material + Autor + Thema (Multi-JOIN)
[7] Vollständige Suche mit Filter (Multi-JOIN)
[8] Alle 7 Abfragen ausführen
[0] Zurück

```

Auswahl: 1

*Materialien pro Themengebiet (COUNT)*

| Themengebiet | Anzahl |
|--------------|--------|
| Informatik   | 5      |

Abbildung 4: Ergebnis von SQL-Abfrage 1 – Materialien pro Themengebiet (COUNT)

Beim Testen ist mir aufgefallen, dass die rich-Tabelle bei sehr langen Titeln den Text abschneidet. Ich habe das durch eine angepasste Spaltenbreite in der Tabellenformatierung gelöst. Insgesamt bin ich mit dem Ergebnis zufrieden – alle geforderten Funktionen sind umgesetzt und die Anwendung verhält sich stabil.

## 4. Funktionsübersicht

Die Anwendung bietet folgende acht Funktionen, die alle über das Hauptmenü erreichbar sind:

| Funktion                | Beschreibung                                                                                                                                  | Menü |
|-------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|------|
| Material hochladen      | Eine Datei wird mit Titel, Thema und Autor versehen und gespeichert – unter 1 MB direkt in der Datenbank, größere Dateien im Ordner uploads/. | [1]  |
| Material herunterladen  | Ein Material wird anhand der ID ausgewählt und in einen lokalen Ordner exportiert.                                                            | [2]  |
| Materialien suchen      | Die Liste lässt sich nach Titel, Themengebiet oder Autor filtern. Ergebnisse erscheinen als formatierte Tabelle.                              | [3]  |
| Material löschen        | Ein Material wird anhand der ID aus der Datenbank entfernt. Liegt die Datei im Dateisystem, wird sie dort ebenfalls gelöscht.                 | [4]  |
| Kommentare verwalten    | Kommentare zu einem Material können angezeigt, hinzugefügt oder gelöscht werden.                                                              | [5]  |
| Listen & Übersichten    | Materialien, Benutzer, Themen, Kategorien und Tags lassen sich vollständig als Tabelle einsehen.                                              | [6]  |
| Datenbank-Abfragen      | Sieben SQL-Abfragen mit Joins und Aggregationen können einzeln oder alle auf einmal ausgeführt werden.                                        | [7]  |
| Neue Einträge erstellen | Benutzer, Themengebiete, Kategorien und Tags lassen sich direkt über das Menü anlegen.                                                        | [8]  |

## 5. Projektabschluss

### 5.1 Soll-Ist-Vergleich

Alle geforderten Funktionen wurden umgesetzt: die acht Menüpunkte, die sieben SQL-Abfragen, die duale Speicherstrategie und die vollständige Datenbankstruktur in 3NF. Die Anforderungen aus Kapitel 1.2 wurden damit vollständig erfüllt.

Nicht umgesetzt wurde eine grafische Oberfläche – das war aber auch keine Anforderung. Eine mögliche Erweiterung wäre eine einfache Web-GUI mit Flask, die ich bei mehr Zeit als Nächstes umsetzen würde.

### 5.2 Zeitaufwand-Vergleich

Insgesamt habe ich 43 Stunden benötigt, geplant waren 35 Stunden – eine Überschreitung von 8 Stunden bzw. etwa 23 %. Die größte Abweichung gab es bei der Python-Implementierung: statt 15 Stunden habe ich 18 Stunden gebraucht. Das lag hauptsächlich an dem bereits erwähnten Problem mit der SQLAlchemy automap-Funktion und an mehreren kleineren Bugs beim Datei-Upload.

Im Nachhinein hätte ich die Zeitplanung realistischer einschätzen können. Für zukünftige Projekte werde ich bei unbekannten Bibliotheken mehr Pufferzeit einplanen.

### 5.3 Persönliche Reflexion

Was gut funktioniert hat: Die Entscheidung, zuerst die Datenbank zu entwerfen und erst dann zu programmieren, war richtig. Das Konsolenmenü ist übersichtlich, und die Trennung in drei Python-Dateien (database.py, services.py, main.py) macht den Code gut wartbar.

Wenn ich das Projekt noch einmal machen würde, fange ich früher mit dem Testen an. Ich habe zu lange darauf gewartet, bis alles "fertig" war – dabei hätte ich durch frühere Tests einige Fehler schneller gefunden. Die SQL-Abfragen hätte ich außerdem zuerst direkt in DBeaver ausprobieren sollen, bevor ich sie in Python übersetzt habe. Das hätte mir mindestens zwei, drei Debugging-Runden erspart.

Der gesamte Quellcode und der überwiegende Teil der Dokumentation wurden eigenständig verfasst. Da Deutsch nicht meine Muttersprache ist, habe ich bei einzelnen Formulierungen meinen Ausbilder gefragt und gelegentlich nachgeschlagen – inhaltlich und technisch liegt das Projekt vollständig bei mir.

## 6. Quellenverzeichnis

Die folgenden Quellen wurden bei der Entwicklung und Dokumentation dieses Projekts verwendet:

[1] Python Software Foundation: Python 3.12 Dokumentation. Online verfügbar unter: <https://docs.python.org/3/> (Zugriff: 16.05.2026).

[2] SQLAlchemy Authors: SQLAlchemy 2.0 Dokumentation. Online verfügbar unter: <https://docs.sqlalchemy.org/en/20/> (Zugriff: 16.05.2026).

[3] Oracle Corporation: MySQL 8.0 Reference Manual. Online verfügbar unter: <https://dev.mysql.com/doc/refman/8.0/en/> (Zugriff: 16.05.2026).

[4] Textualize Ltd.: rich – Python library for rich text and formatting. Online verfügbar unter: <https://rich.readthedocs.io/> (Zugriff: 16.05.2026).



## Anhangsverzeichnis

Die folgenden Anhänge enthalten die technischen Details des Projekts, die den Lesefluss des Berichts unterbrochen hätten, wenn sie direkt in den Haupttext integriert worden wären. Alle Anhänge werden im Bericht mit "(siehe Anhang X)" referenziert.

| Bezeichnung | Inhalt                                                            |
|-------------|-------------------------------------------------------------------|
| Anhang I    | SQL-Schema (vollständige Datenbankdefinition)                     |
| Anhang II   | ERD und ERM (Datenbankdiagramm und Entitätsbeschreibungen)        |
| Anhang III  | Normalisierte Tabellenstruktur (alle 8 Tabellen)                  |
| Anhang IV   | Implementierungsdetails (Architektur, Bibliotheken, SQL-Abfragen) |
| Anhang V    | Installationsanleitung                                            |

## Anhang I: SQL-Schema

Das folgende SQL-Skript erstellt die Datenbank lernmaterialverwaltung vollständig neu: Datenbank, alle acht Tabellen mit Spalten, Datentypen, Primär- und Fremdschlüsseln, sowie zwei Trigger für die automatische Aktualisierung von Zeitstempeln.

```
DROP DATABASE IF EXISTS lernmaterialverwaltung;

CREATE DATABASE lernmaterialverwaltung
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_unicode_ci;

USE lernmaterialverwaltung;

CREATE TABLE themengebiet (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  name        VARCHAR(100) NOT NULL UNIQUE,
  beschreibung TEXT
);

CREATE TABLE benutzer (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  username    VARCHAR(100) NOT NULL UNIQUE,
  email       VARCHAR(255) NOT NULL UNIQUE,
  rolle       ENUM('Lehrer', 'Azubi', 'Admin') NOT NULL
              DEFAULT 'Azubi',
  created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE kategorie (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  name        VARCHAR(100) NOT NULL UNIQUE,
  beschreibung TEXT
);

CREATE TABLE tag (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  name        VARCHAR(50) NOT NULL UNIQUE
);

CREATE TABLE material (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  titel       VARCHAR(255) NOT NULL,
  beschreibung TEXT,
  themengebiet_id INT NOT NULL,
  benutzer_id  INT NOT NULL,
  kategorie_id INT NOT NULL,
  erstellungsdatum TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  aenderungsdatum TIMESTAMP DEFAULT CURRENT_TIMESTAMP
  ON UPDATE CURRENT_TIMESTAMP,
```

```
dateiname          VARCHAR(255),
dateityp           VARCHAR(50),
dateigroesse       BIGINT DEFAULT 0,
speicherort        VARCHAR(500),
inhalt             LONGBLOB,
is_in_database     BOOLEAN DEFAULT FALSE,

FOREIGN KEY (themengebiet_id) REFERENCES themengebiet(id) ON DELETE RESTRICT,
FOREIGN KEY (benutzer_id) REFERENCES benutzer(id) ON DELETE RESTRICT,
FOREIGN KEY (kategorie_id) REFERENCES kategorie(id) ON DELETE RESTRICT
);

CREATE TABLE kommentar (
    id              INT AUTO_INCREMENT PRIMARY KEY,
    material_id     INT NOT NULL,
    autor_id        INT NOT NULL,
    text            TEXT NOT NULL,
    erstellungsdatum TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    aenderungsdatum TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

    FOREIGN KEY (material_id) REFERENCES material(id) ON DELETE CASCADE,

    FOREIGN KEY (autor_id) REFERENCES benutzer(id) ON DELETE RESTRICT
);

CREATE TABLE material_tag (
    id              INT AUTO_INCREMENT PRIMARY KEY,
    material_id     INT NOT NULL,
    tag_id          INT NOT NULL,

    FOREIGN KEY (material_id) REFERENCES material(id) ON DELETE CASCADE,
    FOREIGN KEY (tag_id) REFERENCES tag(id) ON DELETE CASCADE
);

CREATE TABLE version (
    id              INT AUTO_INCREMENT PRIMARY KEY,
    material_id     INT NOT NULL,
    autor_id        INT NOT NULL,
    version_nr      INT NOT NULL,
    aenderungsdatum TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    aenderungen     TEXT,

    FOREIGN KEY (material_id) REFERENCES material(id) ON DELETE CASCADE,

    FOREIGN KEY (autor_id) REFERENCES benutzer(id) ON DELETE RESTRICT
);

DELIMITER //

CREATE TRIGGER before_material_update
```

```
BEFORE UPDATE ON material
FOR EACH ROW
BEGIN
    SET NEW.aenderungsdatum = CURRENT_TIMESTAMP;
END //

CREATE TRIGGER before_kommentar_update
BEFORE UPDATE ON kommentar
FOR EACH ROW
BEGIN
    SET NEW.aenderungsdatum = CURRENT_TIMESTAMP;
END //

DELIMITER ;
```

## Anhang II: ERD und ERM

Das folgende Entity-Relationship-Diagramm wurde mit DBeaver aus der bestehenden MySQL-Datenbank generiert. Es zeigt alle acht Tabellen mit Spalten, Datentypen, Primärschlüsseln (fett markiert) und Fremdschlüsselbeziehungen (Linien mit Kardinalitätsangaben).

Die Linien im Diagramm entsprechen den Fremdschlüsselbeziehungen aus dem Schema: material referenziert benutzer, themengebiet und kategorie; material\_tag verbindet material und tag in einer N:M-Beziehung; kommentar und version referenzieren jeweils material und benutzer.

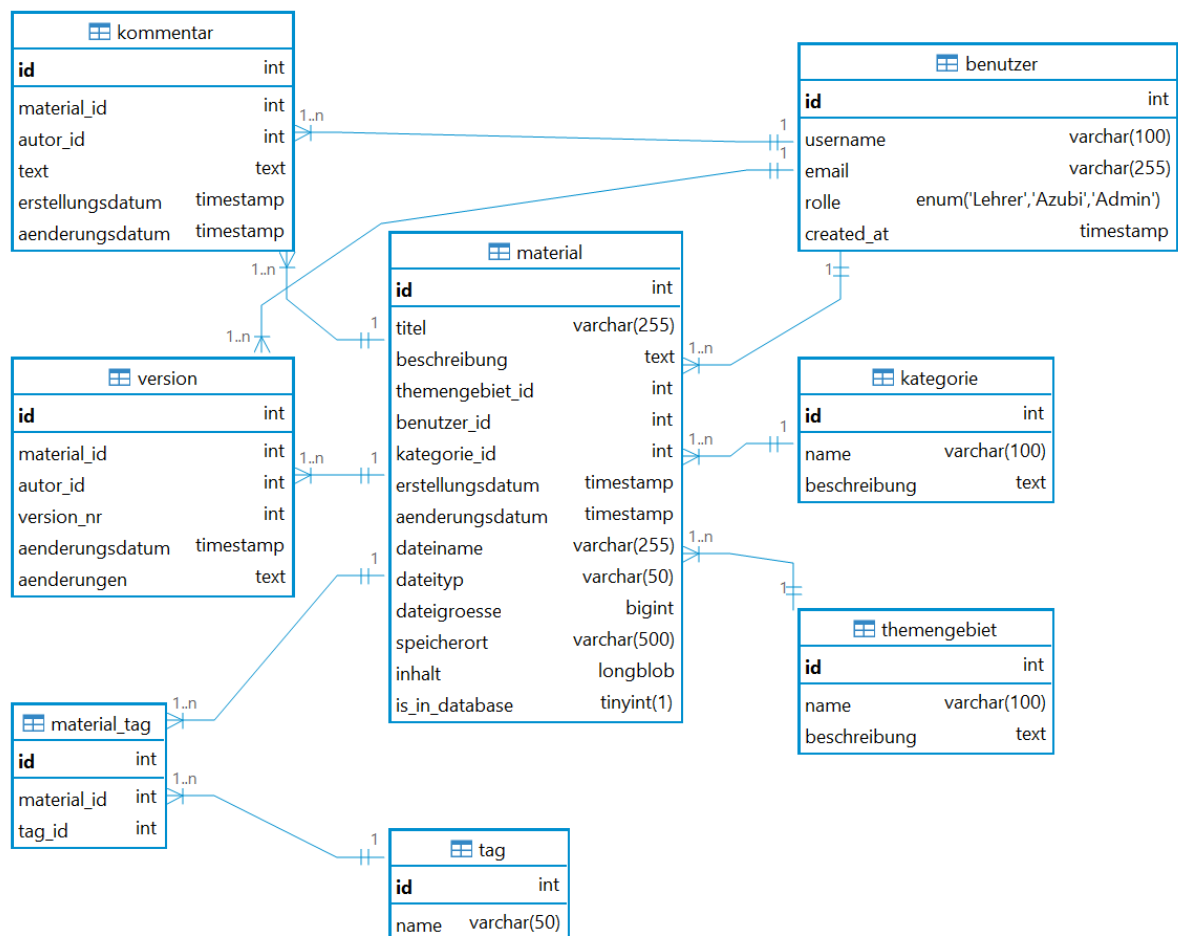


Abbildung 1: ERD der Datenbank lernmaterialverwaltung (generiert mit DBeaver)

### Entitäten und Beziehungen

#### benutzer – Benutzerinformationen

Attribute: id (PK, AI), username (UNIQUE), email (UNIQUE), rolle (Lehrer/Azubi/Admin), created\_at

Beziehungen: 1:N zu material, 1:N zu kommentar, 1:N zu version.

**themengebiet – Fachgebiete der Materialien**

Attribute: id (PK, AI), name (UNIQUE), beschreibung

Beziehungen: 1:N zu material.

**kategorie – Feinkategorisierung**

Attribute: id (PK, AI), name (UNIQUE), beschreibung

Beziehungen: 1:N zu material.

**tag – Freie Schlagwörter**

Attribute: id (PK, AI), name (UNIQUE)

Beziehungen: N:M zu material über material\_tag.

**material – Zentrales Objekt der Anwendung**

Attribute: id, titel, beschreibung, themengebiet\_id (FK), benutzer\_id (FK), kategorie\_id (FK), erstellungsdatum, aenderungsdatum, dateiname, dateityp, dateigroesse, speicherort, inhalt (LONGBLOB), is\_in\_database

Beziehungen: Gehört zu 1 themengebiet, 1 benutzer, 1 kategorie. Hat 1:N zu kommentar und version. N:M zu tag über material\_tag.

**kommentar – Rückmeldungen zu Materialien**

Attribute: id (PK, AI), material\_id (FK), autor\_id (FK), text, erstellungsdatum, aenderungsdatum

Beziehungen: Gehört zu 1 material und 1 benutzer.

**material\_tag – Verbindungstabelle (N:M)**

Attribute: id (PK, AI), material\_id (FK), tag\_id (FK)

Beziehungen: Verbindet material und tag in einer N:M-Beziehung.

**version – Versionshistorie**

Attribute: id (PK, AI), material\_id (FK), autor\_id (FK), version\_nr, aenderungsdatum, aenderungen

Beziehungen: Gehört zu 1 material und 1 benutzer.

## Anhang III: Normalisierte Tabellenstruktur

Alle acht Tabellen liegen in der dritten Normalform (3NF) vor. Jede Spalte ist direkt vom Primärschlüssel abhängig – es gibt weder partielle noch transitive Abhängigkeiten.

Erste Normalform (1NF): Alle Spalten enthalten atomare Werte. Tags sind nicht als kommaseparierte Liste gespeichert, sondern über die Verbindungstabelle material\_tag referenziert. Zweite Normalform (2NF): Alle Tabellen haben einspaltige Primärschlüssel (id), daher gibt es keine partiellen Abhängigkeiten. Dritte Normalform (3NF): Alle Fremdschlüssel (benutzer\_id, themengebiet\_id, kategorie\_id) zeigen auf eigene Tabellen – keine transitiven Abhängigkeiten.

**Tabelle: benutzer**

| Spalte     | Datentyp     | Schlüssel | Beschreibung                           |
|------------|--------------|-----------|----------------------------------------|
| id         | INT          | PK, AI    | Eindeutige Benutzer-ID                 |
| username   | VARCHAR(100) | UNIQUE    | Benutzername                           |
| email      | VARCHAR(255) | UNIQUE    | E-Mail-Adresse                         |
| rolle      | ENUM         |           | Rolle: Lehrer, Azubi oder Admin        |
| created_at | TIMESTAMP    |           | Erstellungsdatum (automatisch gesetzt) |

**Tabelle: themengebiet**

| Spalte       | Datentyp     | Schlüssel | Beschreibung           |
|--------------|--------------|-----------|------------------------|
| id           | INT          | PK, AI    | Eindeutige Thema-ID    |
| name         | VARCHAR(100) | UNIQUE    | Name des Themengebiets |
| beschreibung | TEXT         |           | Optionale Beschreibung |

**Tabelle: kategorie**

| Spalte       | Datentyp     | Schlüssel | Beschreibung            |
|--------------|--------------|-----------|-------------------------|
| id           | INT          | PK, AI    | Eindeutige Kategorie-ID |
| name         | VARCHAR(100) | UNIQUE    | Name der Kategorie      |
| beschreibung | TEXT         |           | Optionale Beschreibung  |

**Tabelle: tag**

| Spalte | Datentyp    | Schlüssel | Beschreibung      |
|--------|-------------|-----------|-------------------|
| id     | INT         | PK, AI    | Eindeutige Tag-ID |
| name   | VARCHAR(50) | UNIQUE    | Tag-Bezeichnung   |

**Tabelle: material**

| Spalte           | Datentyp     | Schlüssel | Beschreibung                             |
|------------------|--------------|-----------|------------------------------------------|
| id               | INT          | PK, AI    | Eindeutige Material-ID                   |
| titel            | VARCHAR(255) |           | Titel des Lernmaterials                  |
| beschreibung     | TEXT         |           | Optionale Beschreibung                   |
| themengebiet_id  | INT          | FK        | Verweis auf themengebiet(id)             |
| benutzer_id      | INT          | FK        | Verweis auf benutzer(id) – Autor         |
| kategorie_id     | INT          | FK        | Verweis auf kategorie(id)                |
| erstellungsdatum | TIMESTAMP    |           | Zeitpunkt des Uploads (automatisch)      |
| aenderungsdatum  | TIMESTAMP    |           | Letztes Änderungsdatum (automatisch)     |
| dateiname        | VARCHAR(255) |           | Originaldateiname                        |
| dateityp         | VARCHAR(50)  |           | MIME-Typ (z. B. application/pdf)         |
| dateigroesse     | BIGINT       |           | Dateigröße in Bytes                      |
| speicherort      | VARCHAR(500) |           | Dateipfad (bei Dateisystemspeicherung)   |
| inhalt           | LONGBLOB     |           | Dateiinhalt (bei Dateien unter 1 MB)     |
| is_in_database   | BOOLEAN      |           | TRUE = Inhalt in DB, FALSE = Dateisystem |

**Tabelle: kommentar**

| Spalte           | Datentyp  | Schlüssel | Beschreibung             |
|------------------|-----------|-----------|--------------------------|
| id               | INT       | PK, AI    | Eindeutige Kommentar-ID  |
| material_id      | INT       | FK        | Verweis auf material(id) |
| autor_id         | INT       | FK        | Verweis auf benutzer(id) |
| text             | TEXT      |           | Kommentartext            |
| erstellungsdatum | TIMESTAMP |           | Erstellungszeitpunkt     |
| aenderungsdatum  | TIMESTAMP |           | Letztes Änderungsdatum   |

**Tabelle: material\_tag**

| Spalte      | Datentyp | Schlüssel | Beschreibung              |
|-------------|----------|-----------|---------------------------|
| id          | INT      | PK, AI    | Eindeutige Verbindungs-ID |
| material_id | INT      | FK        | Verweis auf material(id)  |
| tag_id      | INT      | FK        | Verweis auf tag(id)       |

**Tabelle: version**



| Spalte          | Datentyp  | Schlüssel | Beschreibung                              |
|-----------------|-----------|-----------|-------------------------------------------|
| id              | INT       | PK, AI    | Eindeutige Versions-ID                    |
| material_id     | INT       | FK        | Verweis auf material(id)                  |
| autor_id        | INT       | FK        | Verweis auf benutzer(id)                  |
| version_nr      | INT       |           | Versionsnummer                            |
| aenderungsdatum | TIMESTAMP |           | Zeitpunkt der Änderung                    |
| aenderungen     | TEXT      |           | Beschreibung der vorgenommenen Änderungen |

## Anhang IV: Implementierungsdetails

### Architektur

Das Programm ist in drei Schichten aufgeteilt. `database.py` stellt die Verbindung zur MySQL-Datenbank her und lädt alle Tabellen über SQLAlchemy's `automap_base()` automatisch. `services.py` enthält alle Datenbankfunktionen, von einfachen Abfragen bis zu den sieben Auswertungen. `main.py` ist die Benutzeroberfläche: ein Menü, das Eingaben entgegennimmt und die passenden Funktionen aus `services.py` aufruft.

### Verwendete Bibliotheken

SQLAlchemy 2.0 wird als ORM eingesetzt. Statt SQL direkt zu schreiben, werden Python-Objekte genutzt. Die `automap`-Funktion liest bestehende Tabellen automatisch aus, ohne dass Modellklassen manuell definiert werden müssen. PyMySQL 1.1 ist der Treiber, der SQLAlchemy mit dem MySQL-Server verbindet.

rich 15.0 gibt formatierte Tabellen und farbigen Text in der Konsole aus, ohne grafische Oberfläche. `python-dotenv` 1.2 lädt Zugangsdaten aus einer `.env`-Datei, damit Passwörter nicht direkt im Code stehen.

### Speicherstrategie

Beim Upload wird die Dateigröße geprüft. Ist sie kleiner als 1.048.576 Bytes (1 MB), wird der Inhalt als `LONGBLOB` in der Tabelle `material` gespeichert und `is_in_database` auf `TRUE` gesetzt. Größere Dateien werden in den Ordner `uploads/` kopiert, und nur der Pfad wird in der Spalte `speicherort` hinterlegt (`is_in_database = FALSE`). Beim Download entscheidet das System anhand dieses Flags, ob der Inhalt aus der Datenbank gelesen oder die Datei vom Dateisystem kopiert wird.

### Die sieben Datenbankabfragen

Abfrage 1 – Materialien pro Themengebiet: `COUNT`-Aggregation mit `GROUP BY`. Zeigt, wie viele Materialien je Thema gespeichert sind (`JOIN` auf `themengebiet`).

Abfrage 2 – Durchschnittliche Dateigröße pro Dateityp: `AVG`-Aggregation mit `GROUP BY` auf `material.dateityp`.

Abfrage 3 – Materialien mit Themengebiet: `INNER JOIN` zwischen `material` und `themengebiet`.

Abfrage 4 – Materialien mit Kommentaren: `INNER JOIN` zwischen `material` und `kommentar`.

Abfrage 5 – Kommentaranzahl pro Material: `INNER JOIN` mit `COUNT` (`JOIN` + Aggregation kombiniert).

Abfrage 6 – Material mit Autor und Thema: Zwei `JOINS` auf `benutzer` und `themengebiet`.

Abfrage 7 – Vollständige Suche: Drei `JOINS` auf `benutzer`, `themengebiet` und `kategorie`, mit optionalen `WHERE`-Filtern nach Thema oder Autor.

### Quellcode-Kommentare

Alle drei Python-Dateien sind auf Deutsch kommentiert. Jede Funktion hat einen erklärenden Kommentar direkt über der Definition. Wichtige Konstanten und technische Entscheidungen sind ebenfalls dokumentiert. Einige Beispiele:

```
# database.py
# Hier verbinden wir uns mit der MySQL-Datenbank.
# Die Zugangsdaten kommen aus der .env Datei.

# SessionLocal öffnet eine neue Verbindung wenn wir sie brauchen
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)

# SQLAlchemy liest die Tabellen automatisch aus der Datenbank
Base = automap_base()
```

```
# services.py
# Dateien kleiner als 1 MB kommen direkt in die Datenbank
MAX_SIZE = 1 * 1024 * 1024

# Eine Datei hochladen - kleine Dateien in die DB, große auf die Festplatte
def upload_material(db, file_path, titel, beschreibung, ...):

# Abfrage 1: Wie viele Materialien gibt es pro Themengebiet? (COUNT)
def anzahl_material_pro_themen(db):
```

```
# main.py
# Menü zum Hochladen - Benutzer gibt Dateipfad und Infos ein
def upload_menu():

# Das Hauptmenü - hier startet alles
def hauptmenu():
```

## Anhang V: Installationsanleitung

Um die Anwendung lokal auszuführen, werden Python 3.10 (oder neuer), MySQL 8.0 und pip benötigt.

### Schritt 1 – Abhängigkeiten installieren

Im Projektverzeichnis folgenden Befehl ausführen:

```
pip install -r requirements.txt
```

### Schritt 2 – .env Datei anlegen

Im Projektverzeichnis eine Datei mit dem Namen .env erstellen und folgende Werte eintragen:

```
DB_DRIVER=mysql+pymysql
DB_USERNAME=root
DB_PASSWORD=IhrPasswort
DB_HOST=localhost
DB_PORT=3306
DB_NAME=lernmaterialverwaltung
```

### Schritt 3 – Datenbank einrichten

Das SQL-Schema in MySQL importieren (einmalig, erstellt die Datenbank und alle Tabellen):

```
mysql -u root -p < sql/schema.sql
```

### Schritt 4 – Anwendung starten

In den app-Ordner wechseln und das Programm starten:

```
cd app
python main.py
```

Das Hauptmenü erscheint direkt im Terminal. Die Anwendung läuft vollständig textbasiert und benötigt keine grafische Oberfläche.